

give me first outline and what to do a layout

Task 2: LRU Cache, plan and layout

1. Decisions to make first

- **Language:** TypeScript (Node). It matches your stack and is easy to run and test.
- **Core design:** a `Map<key, Node>` plus a **doubly linked list** with dummy head and tail nodes.
- The Map gives $O(1)$ lookup.
- The list gives $O(1)$ move-to-front and $O(1)$ eviction.
- Don't rely on the JS `Map` insertion-order trick. Graders want to see you maintain the order yourself. You can mention the trick in the README as an alternative.
- **Bonus (TTL):** store an `expiresAt` on each node and use lazy expiration. Do it after the core works.

2. Repo layout

```
lru-cache/
├── src/
│   ├── LRUCache.ts      # the implementation
│   └── index.ts          # demo script that prints output (for
│                           screenshot)
├── tests/
│   └── LRUCache.test.ts  # unit tests
├── screenshots/
│   ├── output.png
│   └── ttl-output.png    # if you do the bonus
├── package.json
├── tsconfig.json
└── README.md
```

3. Implementation outline (`LRUCache.ts`)

- `Node { key, value, prev, next, expiresAt? }`
- `constructor(capacity)`: throw if capacity is not a positive integer. Create the head/tail sentinels and link them.
- Private helpers:
- `addToFront(node)`: insert right after head

- `removeNode(node)`: unlink from its neighbors
- `moveToFront(node)`: `removeNode` then `addToFront`
- `get(key)`:
 - Return -1 if the key is missing.
 - Otherwise `moveToFront` and return the value.
- `put(key, value)`:
 - If the key exists, update the value and `moveToFront`.
 - Otherwise create a node, add it to the front and set the Map entry.
 - If `size > capacity`, remove `tail.prev` and delete its key from the Map.

4. Tests to cover

- The exact example from the task (A, B, C with capacity 2)
- `get` on a missing key returns -1
- `put` on an existing key updates the value and refreshes recency
- A successful `get` protects a key from eviction
- Capacity 1 edge case
- Invalid capacity (0, negative) throws
- (Bonus) an entry expires after its TTL, and an expired entry doesn't count toward capacity

5. Demo script (`index.ts`), which is what your screenshot shows

Print a labeled line for every operation so the screenshot proves each requirement:

```
put(A,10) | put(B,20)
get(A) -> 10
put(C,30) --> evicts B
get(B) -> -1
get(C) -> 30
get(A) -> 10
```

Also print the current order (e.g. `state: [A, C]`, most recent first) after each step. Generate this from real code, not typed text. One screenshot of the terminal running `npm start` is enough.

6. README outline

- **Overview:** what it is and the API
- **Data structures and why:** HashMap plus doubly linked list, with a

short reason for each

- **How LRU ordering is maintained:** the head is most recent, the tail is least recent, and each `get/put` moves the node to the front
- **Complexity:** time is $O(1)$ average for `get` and `put`. Space is $O(\text{capacity})$.
- **How to run:** `npm install`, `npm start`, `npm test`
- **Example output:** embed the screenshot
- **Bonus, TTL (if done):**
 - Approach: lazy expiration on `get`, so an expired entry is removed and returns `-1`
 - Trade-offs: expired entries hold memory until accessed or evicted. An optional periodic sweep fixes that but adds a timer and complexity.
 - Screenshot: use a short TTL and a `setTimeout` to show the expiry

7. Order of work

- Set up the repo and TypeScript
- Write `LRUCache.ts`
- Write the tests and get them passing
- Write `index.ts` and take the screenshot
- Write the README
- Push to GitHub and check that the README renders and the screenshot displays
- Add TTL last, only if time allows

Want me to write the full `LRUCache.ts`, the tests and the demo script next?

ok what should be the to build the project?

Build steps: setup to running

1. Create the project

```
mkdir lru-cache && cd lru-cache
git init
npm init -y
npm install -D typescript tsx vitest @types/node
npx tsc --init
mkdir src tests screenshots
```

`tsx` runs TypeScript directly, and `vitest` is a light test runner. That means no build step is needed for the demo.

2. Configure `tsconfig.json`

Keep it minimal:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "outDir": "dist",
    "skipLibCheck": true
  },
  "include": ["src", "tests"]
}
```

3. Add scripts to `package.json`

```
"scripts": {
  "start": "tsx src/index.ts",
  "test": "vitest run",
  "build": "tsc"
}
```

4. Build in this order

- `src/LRUCache.ts`: the Node class, the sentinels, and the `get/put` helpers.
- `tests/LRUCache.test.ts`, then run `npm test` until everything passes.
- `src/index.ts`, then run `npm start`. This is your screenshot source.
- Write `README.md` last, once you know the real output.
- Take the screenshot of the terminal after `npm start`.
- Add `.gitignore` with `node_modules` and `dist`.

5. Push to GitHub

```
git add .
git commit -m "Implement LRU cache with tests and README"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

6. Final checks before submitting

- Clone the repo into a fresh folder and run `npm install && npm start && npm test`. This proves the run instructions work.
- Confirm the README renders and the screenshot displays on GitHub.
- Make sure the screenshot shows put, get, eviction, and the returned values, with the terminal command visible.
- Make sure the repo is public, or accessible to the graders.

If you meant something else by "the to build," such as a prompt to give an AI, or the build order for a different part, tell me. Otherwise, want me to write `LRUCache.ts`, the tests, and the demo script next?

now give me `LRUCache.ts` and `index.ts`

Both files are above. I ran the demo and it matches the task's example: `get("B")` returns -1 after `put("C", 30)` evicts B, and the final `get("A")` returns 10.

A few things worth knowing:

- `put()` returns the evicted key, or `null` if nothing was evicted. The demo uses this to print `--> evicted "B"`, and it doesn't change the `put(key, value)` API.
- `keys()` returns keys from most to least recently used. The demo uses it to print the state after every step, so the screenshot shows the ordering.
- I added a small extra section to the demo that updates an existing key. Delete it if you want the output to match the task example exactly.

Take the screenshot from your own terminal with `npm start`, with the command visible, since the task requires real output from your run.

Next up are the tests (`tests/LRUCache.test.ts`), and then the README. Want me to write the tests now?



LRUCache

TS · Click to download



Index

TS · Click to download

give me a gitignore file

Here's a `.gitignore` for this project:

`node_modules`

```
dist
.env
.DS_Store
*.log
coverage
```

Create it in the repo root (`lru-cache/.gitignore`), or with one command:

```
printf "node_modules\ndist\n.env\n.DS_Store\n*.log\ncoverage\n" >
.gitignore
```

Don't ignore the `screenshots/` folder, since the README needs those images on GitHub. If you already ran `git add .` before creating this file, run `git rm -r --cached node_modules` to untrack it.

Want the tests next?

```
git remote add origin
```

<https://github.com/FahimFaysalNirjhar/Task-2-LRU-Cache-plan-and-layout.git>

error: remote origin already exists.

The `origin` remote is already set in this folder. Check where it currently points:

```
git remote -v
```

If it points to the wrong repo (for example, an earlier project's repo), change it to the new one:

```
git remote set-url origin
https://github.com/FahimFaysalNirjhar/Task-2-LRU-Cache-plan-and-l
ayout.git
```

If it already points to the right URL, you don't need to add it again. Go straight to the push:

```
git branch -M main
git push -u origin main
```

If `git remote -v` shows a different repo, you may have created this folder inside another project's repo, such as your Task 1 folder. Run `git rev-parse --show-toplevel` to see which folder Git treats as the repo root. If it's not your `lru-cache` folder, run `git init` inside `lru-cache` and then add the remote there.

The repo name `Task-2-LRU-Cache-plan-and-layout` is a bit awkward for a submission. If you haven't shared it yet, you can rename it on GitHub under Settings, or create a new repo called `lru-cache` and set the remote to that one.

```
"compilerOptions": {
  "target": "ES2020",
  "module": "commonjs",
  "strict": true,
  "esModuleInterop": true,
  "outDir": "dist",
  "skipLibCheck": true
},
"include": ["src", "tests"]
}
```

The common source directory of 'tsconfig.json' is './src'. The 'rootDir' setting must be explicitly set to this or another path to adjust your output's file layout. Visit <https://aka.ms/ts6> for migration information.

That's TypeScript 6 behavior: it no longer infers the output root from your source files, so `rootDir` has to be set explicitly. Since `tsc` only needs to build the source, use this:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "rootDir": "./src",
    "outDir": "dist",
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

Two changes: `rootDir` is now `./src`, and `include` no longer lists `tests`. Your tests still run fine because `vitest` compiles them itself and doesn't go through `tsc`.

If you'd rather have `tsc` type-check the tests too, keep `"include": ["src", "tests"]` and set `"rootDir": "."` instead. The build output then goes to `dist/src` and `dist/tests`.

`npm start` uses `tsx`, so it works either way. This only affects `npm run build`.

```
PS C:\Users\User\Desktop\Next_Level\lru-cache> git remote add origin
https://github.com/FahimFaysalNirjhar/LRU-Cache.git
error: remote origin already exists.
```

Same cause as before: `origin` is already set (probably to the earlier `Task-2-LRU-Cache-plan-and-layout` URL). Point it at the new repo instead of adding it:

```
git remote set-url origin
https://github.com/FahimFaysalNirjhar/LRU-Cache.git
git remote -v
```

`git remote -v` should now show the `LRU-Cache.git` URL for both fetch and push. Then push:

```
git branch -M main
git push -u origin main
```

If the push is rejected with a message like "fetch first" or "unrelated histories", the GitHub repo was created with a README or license. If you don't need what's on GitHub, overwrite it with:

```
git push -u origin main --force
```

Only use `--force` on a brand-new repo like this one, since it replaces whatever is on the remote.

here the issue is `node_modules` is uploaded in git i mistakenly added in my fist git repo after i deleted that repo still it show

Deleting the GitHub repo doesn't touch your local Git. The `node_modules` files are still tracked in your local index and in your commit history, so they get pushed to the new repo too. Adding `.gitignore` afterward doesn't help for files Git is already tracking.

Since this is a brand-new repo with only a few commits, the cleanest fix is to start the history fresh. In PowerShell, from the `lru-cache` folder:

```
# 1. make sure .gitignore exists and contains node_modules and dist
Get-Content .gitignore

# 2. delete the old Git history (this does not touch your files)
Remove-Item -Recurse -Force .git

# 3. start over
git init
git add .
git status
```

Before committing, check that `git status` doesn't list anything under `node_modules/`. If it looks right:

```
git commit -m "Implement LRU cache with tests and README"
git branch -M main
git remote add origin
https://github.com/FahimFaysalNirjhar/LRU-Cache.git
git push -u origin main --force
```

The `--force` is needed because the GitHub repo may already contain the earlier push. Since you wiped the local history, this replaces it.

If you'd rather keep your commit history, run this instead:

```
git rm -r --cached node_modules
git commit -m "Remove node_modules from tracking"
```

That stops tracking `node_modules` going forward, but the files stay in old commits, so the repo will still be large and the GitHub history will still contain them. For a graded submission, the fresh start is better.

Afterward, open the repo on GitHub and confirm there's no `node_modules` folder, and that `screenshots/` and the README are there.

`LRUCache.test.ts`

The test file is above. I ran it against your `LRUCache.ts` and all 12 tests pass. It covers:

- The exact example from the task
- A missing key returning -1
- Updating an existing key, and that update refreshing recency
- A successful `get` protecting a key from eviction
- The evicted key being reported by `put`
- Capacity never being exceeded
- The capacity-1 edge case
- MRU-to-LRU ordering, and a failed `get` not changing the order
- A stored value of `0` (a check that `0` isn't treated as a miss)
- Invalid capacities (0, -1, 1.5, NaN) throwing

Save it as `tests/LRUCache.test.ts`, then run `npm test`. Take a second screenshot of that passing run to include in the README as extra proof.

There are no TTL tests yet because the TTL bonus isn't implemented. If you decide to do it, I can add the TTL support and its tests.

The README is the last required piece. Want me to write it next?



LRUCache.test
TS · Click to download

give me readme.md and use instruction in provided image

The README is above. It covers everything the required part of the submission asks for: the data structures and why, how LRU ordering is maintained, time and space complexity, and how to run it.

Before you push:

- **Screenshots:** it links `screenshots/output.png` (from `npm start`) and `screenshots/tests.png` (from `npm test`). Add both files with exactly those names, or the images will show as broken on GitHub. The output

block in the README is the real output from my run of your code, but the screenshot itself has to come from your terminal.

- **Repo URL:** the clone command uses `https://github.com/FahimFaysalNirjhar/LRU-Cache.git`. Change it if your repo name differs.
- **Demo output:** the README shows only the main demo section. Your `index.ts` also prints the extra "update existing key" section, so remove that section from `index.ts` if you want the screenshot to match the README exactly.
- **TTL bonus:** I left it out because it isn't implemented, and the task says to include it only if done. If you want the bonus, I can add TTL support to `LRUCache.ts`, plus a demo, tests, and a README section on the approach and trade-offs.

Once the screenshots are in, push to GitHub, open the repo page, and check that the README renders and both images display.



README

MD · Click to download

[Image]

Both screenshots show real output from your implementation, and they cover what the task asks for: put and get operations, the eviction (`evicted "B"`), the returned values (10, -1, 30, 10), and the LRU order after each step.

Demo screenshot: I'd retake it. The crop starts at `=== LRU Cache demo`, so the command that produced it isn't visible. Since graders need proof the output is real, include the prompt line at the top:

```
PS C:\Users\User\Desktop\Next_Level\lru-cache> npm start
```

Scroll up a little and capture from that line down, like you did for the test screenshot.

Test screenshot: this one is good, since it shows the `npm test` command and 12 passed. The right edge cuts off some millisecond timings, so a slightly wider crop would look cleaner, but that's cosmetic.

Save them:

- Demo screenshot as `screenshots/output.png`
- Test screenshot as `screenshots/tests.png`

Those are the exact names the README links to.

Extra section: your screenshot includes the "update existing key" section, which the README's example block doesn't show. That's fine, since the

screenshot proves more, not less. If you'd rather they match exactly, add those last lines to the README block.

Then run `git add .`, commit, and push. Once it's up, check on GitHub that both images display in the README. I can go through a final submission checklist with you if you'd like.